CreditCardPayment.java - Deep Dive Explanation

1. OVERVIEW & LINKAGE

This is a Concrete Subclass. It links to Payment by extending it, inheriting its base fields (amount, ID). It links to PaymentProcessor indirectly—when the processor calls the generic process() method, this class's specific override is what actually executes.

2. DETAILED CODE BREAKDOWN

```
private String cardNumber;
private String cvv;
```

These fields are "private", meaning only this specific class knows they exist. The Payment base class doesn't know about them, and the PaymentProcessor doesn't know about them. This ensures data isolation.

```
super(amount, currency);
```

Inside the constructor, this line passes the generic data (amount and currency) UP to the parent Payment class so it can handle the ID generation and timestamping. Then the constructor saves the card-specific data.

```
@Override
public boolean process(...) { ... }
```

The @Override annotation tells the Java compiler we are fulfilling the abstract contract from the parent class.

```
String digits = cardNumber.replaceAll("\\s", "");
boolean validCard = digits.length() >= 12 && cvv.matches("^\\d{3}$");
```

Algorithm Step 1 (Validation): It strips spaces from the card number and ensures it is at least 12 digits long. It also uses a Regular Expression (regex) to ensure the CVV is exactly 3 numbers. If it fails, it sets this.status = "FAILED" and aborts.

```
boolean authorized = amount <= 500_000;
```

Algorithm Step 2 (Authorization): It simulates a network connection to a bank. We implemented an artificial rule that any transaction over $500,000 gets declined. If authorized, it sets this.status = "SUCCESS" and returns true to the processor.